

PROJECT REPORT

Heart Disease Risk Analytics System

1. Application Overview

This project builds a small cloud-based system for analysing a medical dataset (heart disease data). I used Hadoop for distributed storage and Hive for SQL queries. The system also includes two web pages: one shows statistics in charts, and another lets users calculate their own risk score.

The system performs the following core tasks:

- Loads heart disease data (1025 records) into Hadoop's distributed file system (HDFS)
- Cleans and processes data using Hive SQL queries
- Calculates risk scores (0-5) for each patient based on five clinical factors
- Groups data by age, sex, and risk level to find patterns
- Exports results as CSV files for visualization
- Shows results in a web dashboard with interactive charts
- Provides a risk calculator where users input their own health data

The project demonstrates how distributed computing tools (Hadoop + Hive) can handle medical data at scale while providing practical tools for doctors and patients.

2. Objectives

The project goals:

- **Data ingestion:** Successfully upload the dataset (heart.csv) into HDFS.
- **Schema creation:** Create Hive healthcare_dw database and external tables.
- **Data cleaning :** Apply Hive SQL preprocessing.
- **Risk scoring:** Apply a rule-based scoring model (0–5 scale).
- **Analytics:** Compute statistics by age, sex, and risk level.
- **Export results:** Save results to CSV for python dashboard
- **Web dashboard:** Build a dashboard using HTML and Chart.js.
- **Interactive calculator:** Built risk scoring tool where users input their data

All objectives were completed.

3. Problem Description and Dataset Collection

3.1 Dataset Overview

The medical dataset used in this project is the Heart Disease dataset, containing 1026 patient records with 14 clinical features including demographic, behavioural, and clinical features associated with cardiac health.

Dataset fields:

Field	Description	Type
age	Patient age in years	Integer (29-77)
sex	1 = male, 0 = female	Binary
cp	Chest pain type (0-3)	Integer
trestbps	Resting blood pressure (mm Hg)	Integer (94-200)
chol	Cholesterol level (mg/dl)	Integer (126-564)
fbs	Fasting blood sugar > 120 mg/dl	Binary
restecg	Resting ECG results (0-2)	Integer
thalach	Maximum heart rate	Integer (71-202)
exang	Exercise-induced angina	Binary
oldpeak	ST depression	Float
slope	ST segment slope	Integer
ca	Number of major vessels (0-3)	Integer
thal	Thalassemia	Integer
target	Heart disease present (0-4)	Integer

Problem being solved:

Medical professionals and researchers often need:

- **Storing large clinical datasets:** Traditional databases struggle with millions of patient records
- **Running complex analyses:** Need to find patterns across demographics
- **Consistent risk assessment:** Manual scoring is slow and inconsistent
- **Quick decision support:** Doctors need instant risk calculations
- **Data visualization:** Non-technical staff need charts, not tables

This project shows how cloud data tools can solve these problems. Even though my chosen dataset is small (1025 records), the workflow scales to millions of records because Hadoop distributes work across multiple machines

4. Methodology and Implementation

4.1 Hadoop Cluster Setup

A multi-container Hadoop cluster (NameNode, DataNode, ResourceManager, NodeManager, HistoryServer) was deployed using Docker. This simulates a distributed environment.

Cluster components:

- **NameNode:** Manages file metadata, coordinates data distribution
- **DataNode:** Stores actual data blocks
- **ResourceManager:** Allocates computing resources for jobs
- **NodeManager:** Runs tasks on worker nodes
- **HistoryServer:** Tracks completed jobs for debugging

This setup gave me a working Hadoop 3.2.1 cluster running on my laptop:

☐	▼	●	docker-hadoop	-	-	-	6.08%	17 hours ago
☐		●	namenode	b94bbdba024e	bde2020/hadoop-namenode:2.0.0-hado	9000:9000 c* Show all ports (2)	4.39%	17 hours ago
☐		●	nodemanager	5f5d031c81bc	bde2020/hadoop-nodemanager:2.0.0-ha		0.52%	17 hours ago
☐		●	datanode	627eda94f7a2	bde2020/hadoop-datanode:2.0.0-hadoo		0.31%	17 hours ago
☐		●	resourcemanager	e7339097ca07	bde2020/hadoop-resourcemanager:2.0.		0.74%	17 hours ago
☐		●	historyserver	830a9aa73b94	bde2020/hadoop-historyserver:2.0.0-ha		0.12%	17 hours ago

4.2 Data Ingestion into HDFS

The CSV file (heart.csv) was uploaded to HDFS:

Created directory structure

```
hdfs dfs -mkdir -p /user/root/healthcare/heart
```

Uploaded data file

```
hdfs dfs -put heart.csv /user/root/healthcare/heart/
```

HDFS automatically splits the file into blocks (default 128MB). For larger datasets, these blocks get distributed across multiple DataNodes. This enables parallel processing.

4.3 Hive Configuration

Hive lets you query HDFS data using SQL instead of writing MapReduce code. I installed Hive 3.1.2 and configured it to use Derby database for metadata.

Entered NameNode container

```
docker exec -it namenode bash
```

Initialized metadata database

```
cd /opt/hive
```

```
schematool -dbType derby -initSchema
```

A new database (healthcare_dw) and an external table were created to read the CSV file.

```
CREATE DATABASE IF NOT EXISTS healthcare_dw;
```

```
USE healthcare_dw;
```

```
hive> CREATE DATABASE IF NOT EXISTS healthcare_dw;
USE healthcare_dw;OK
Time taken: 0.072 seconds
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS heart_raw (
```

```
  age      INT,
```

```
  sex      INT,
```

```
  cp       INT,
```

```
  trestbps INT,
```

```
  chol     INT,
```

```

    fbs      INT,
    restecg  INT,
    thalach  INT,
    exang     INT,
    oldpeak  DOUBLE,
    slope    INT,
    ca       INT,
    thal     INT,
    target   INT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/user/root/healthcare/heart'
TBLPROPERTIES ("skip.header.line.count"="1");

```

```

hive> DESCRIBE heart_raw;
OK
age                int
sex                int
cp                 int
trestbps           int
chol               int
fbs                int
restecg            int
thalach            int
exang              int
oldpeak            double
slope              int
ca                 int
thal               int
target             int
Time taken: 0.162 seconds, Fetched: 14 row(s)
hive> SELECT * FROM heart_raw LIMIT 10;
OK
52      1      0      125      212      0      1      168      0      1.0      2      2      3      0
53      1      0      140      203      1      0      155      1      3.1      0      0      3      0
70      1      0      145      174      0      1      125      1      2.6      0      0      3      0
61      1      0      148      203      0      1      161      0      0.0      2      1      3      0
62      0      0      138      294      1      1      106      0      1.9      1      3      2      0
58      0      0      100      248      0      0      122      0      1.0      1      0      2      1
58      1      0      114      318      0      2      140      0      4.4      0      3      1      0
55      1      0      160      289      0      0      145      1      0.8      1      1      3      0
46      1      0      120      249      0      0      144      0      0.8      2      0      3      0
54      1      0      122      286      0      0      116      1      3.2      1      2      2      0
Time taken: 0.276 seconds, Fetched: 10 row(s)

```

4.4 Data Cleaning and Risk Scoring

I wrote Hive SQL to clean the data and calculate risk scores. The scoring model uses simple clinical thresholds that doctors commonly use for screening.

Risk factors (each worth 1 point):

- age > 55
- resting BP > 130
- cholesterol > 250
- fasting blood sugar > 120
- exercise-induced angina = 1

Total score: 0-5 points

```
CASE WHEN age > 55 THEN 1 ELSE 0 END +  
CASE WHEN rest_bp > 130 THEN 1 ELSE 0 END +  
CASE WHEN chol > 250 THEN 1 ELSE 0 END +  
CASE WHEN fbs = 1 THEN 1 ELSE 0 END +  
CASE WHEN exang = 1 THEN 1 ELSE 0 END AS risk_score
```

Risk levels are defined as:

- 0-1 → Low
- 2-3 → Medium
- 4-5 → High

```
CASE  
  WHEN risk_score <= 1 THEN 'Low'  
  WHEN risk_score <= 3 THEN 'Medium'  
  ELSE 'High'  
END AS risk_level
```

These rules are easy to explain and match early-warning checks used in clinical screening.

4.5 Analysis Queries

I ran several SQL queries to find patterns in the data:

Query 1: Disease rate by age group

```
hive> SELECT  
  CASE  
    WHEN age < 40 THEN 'Under 40'  
    WHEN age BETWEEN 40 AND 54 THEN '40-54'  
    WHEN age BETWEEN 55 AND 64 THEN '55-64'  
    ELSE '65+'  
  END AS age_group,  
  
  COUNT(*) AS total,  
  SUM(CASE WHEN target > 0 THEN 1 ELSE 0 END) AS disease_cases,  
  ROUND(100.0 * SUM(CASE WHEN target > 0 THEN 1 ELSE 0 END) / COUNT(*), 1) AS disease_rate  
  
FROM heart_raw  
  
GROUP BY  
  CASE  
    WHEN age < 40 THEN 'Under 40'  
    WHEN age BETWEEN 40 AND 54 THEN '40-54'  
    WHEN age BETWEEN 55 AND 64 THEN '55-64'  
    ELSE '65+'  
  END  
  
ORDER BY age_group;
```

Query result:

```
...  
40-54  419    276    65.9  
55-64  411    141    34.3  
65+    138     67    48.6  
Under 40  57     42    73.7  
Time taken: 39.461 seconds, Fetched: 4 row(s)
```

See the file ([age_group_stats.csv](#))

Query 2: Disease rate by sex

```
hive> > SELECT
    CASE WHEN sex = 1 THEN 'Male' ELSE 'Female' END as gender,
    COUNT(*) as total,
    SUM(CASE WHEN target > 0 THEN 1 ELSE 0 END) as disease_cases,
    ROUND(100.0 * SUM(CASE WHEN target > 0 THEN 1 ELSE 0 END) / COUNT(*), 1) as disease_rate
FROM heart_raw
GROUP BY sex; > > > > > >
```

Query result:

```
Female 312    226    72.4
Male   713    300    42.1
Time taken: 18.942 seconds, Fetched: 2 row(s)
```

See the file (**sex_stats.csv**)

Query 3: Disease rate by age group and sex

This query provides a detailed breakdown of heart disease prevalence across different age groups and sexes. This is critical for identifying high-risk demographics.

```
hive> SELECT
    age_group,
    sex,
    COUNT(*) AS total,
    SUM(CASE WHEN target = 1 THEN 1 ELSE 0 END) AS heart_cases,
    ROUND(100.0 * SUM(CASE WHEN target = 1 THEN 1 ELSE 0 END) / COUNT(*), 2) AS heart_rate_pct
FROM heart_scored
GROUP BY age_group, sex
ORDER BY age_group, sex;
> > > > > > > > Query ID = root_20251203170443_efcb1572-ad30-4ca9-a0
9a-7d5641ca8e17
```

Query result:

```
40-54  0      107    100    93.46
40-54  1      312    176    56.41
55-64  0      132     61    46.21
55-64  1      279     80    28.67
65+    0       56     48    85.71
65+    1       82     19    23.17
Under 40  0       17     17   100.00
Under 40  1       40     25    62.50
Time taken: 37.608 seconds, Fetched: 8 row(s)
```

See the file (**age_sex_stats.csv**)

Query 4: Risk level distribution

```
hive> CREATE TABLE IF NOT EXISTS risk_level_stats AS
SELECT
    risk_level,
    COUNT(*) AS total
FROM heart_scored
GROUP BY risk_level; > > > > >
Query ID = root_20251202185238_18a5e752-6a40-49e7-a1ed-2830dbd2e95a
Total jobs = 1
Launching Job 1 out of 1
```

Query result:

```
High    94
Low     423
Medium  508
Time taken: 16.937 seconds, Fetched: 3 row(s)
```

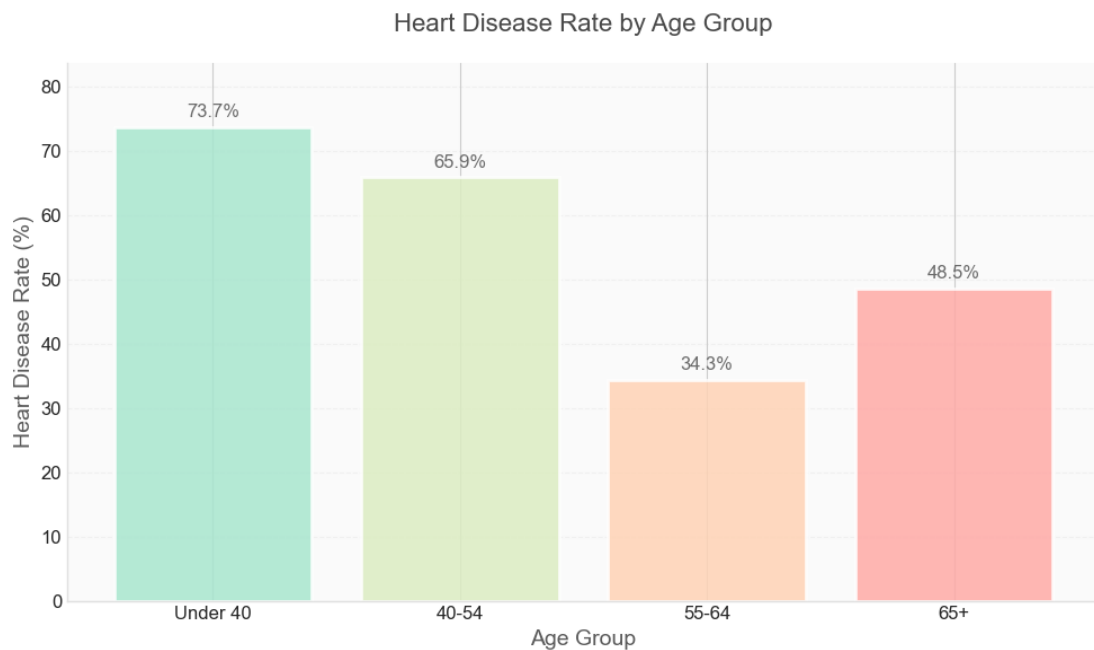
See the file ([risk_level_stats.csv](#))

4.6 Data Visualisation

Two dashboards were created:

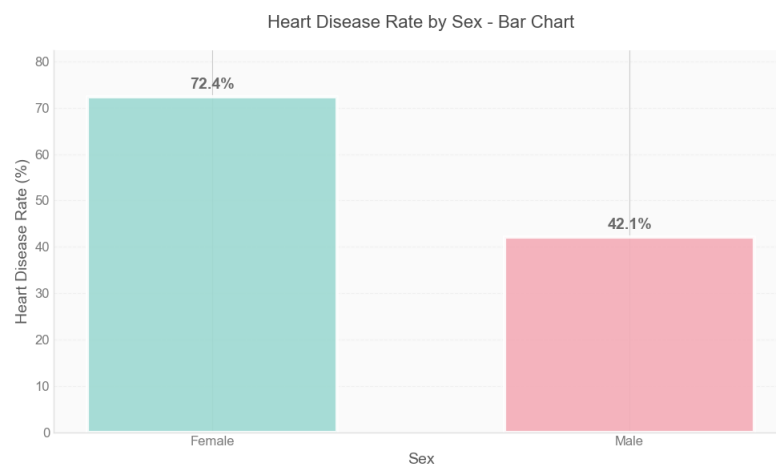
- **Python Notebook (Matplotlib)** - initial exploratory plots based on four csv files.

Bar chart: Heart disease rate by age



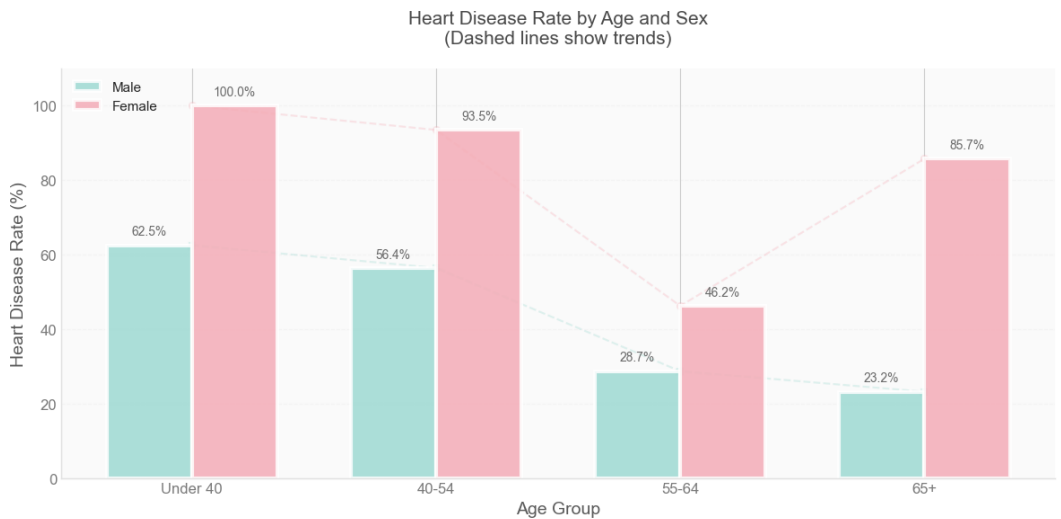
Younger patients show unexpectedly higher disease rates (73.7% for Under 40) compared to older groups (34.3% for 55-64), contradicting typical age-disease patterns.

Bar chart: Heart disease rate by sex



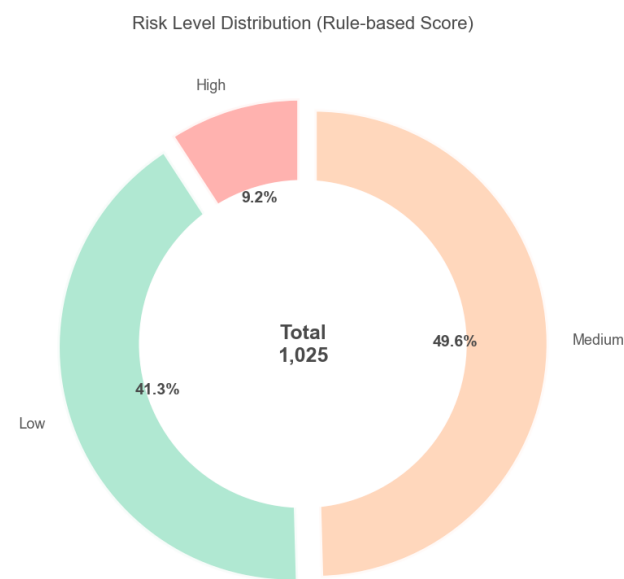
Females demonstrate significantly higher heart disease prevalence (72.4%) than males (42.1%), representing a 30-percentage-point gender gap.

Grouped bar and line chart: Compare male vs female side-by-side for each age group



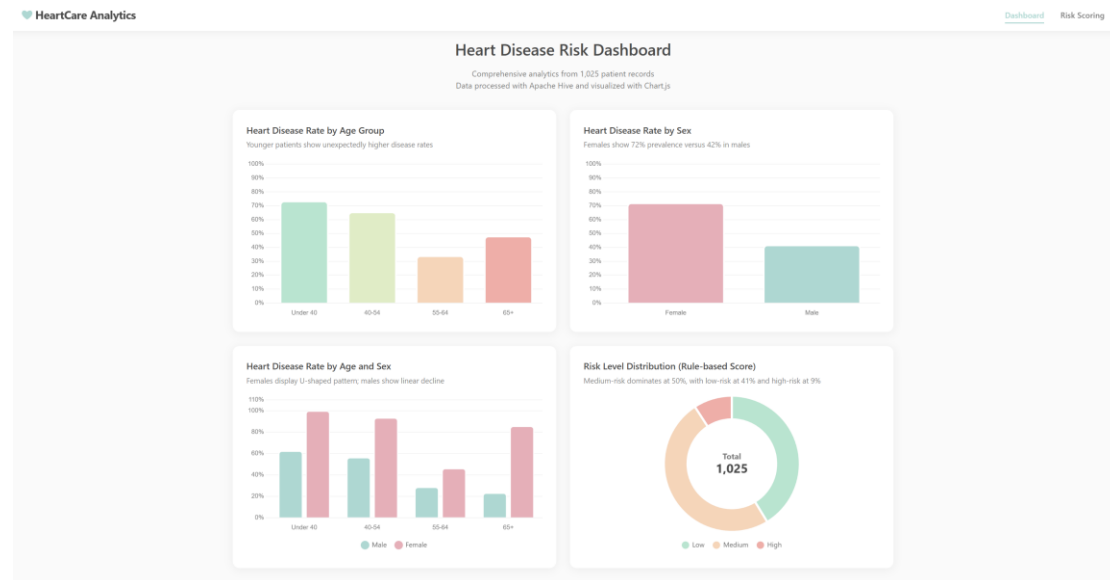
- Females have consistently higher rates in younger ages
- While male rates decline steadily with age (from 62.5% to 23.2%), female rates follow a U-shaped curve, starting very high (100%), dropping in middle age (46.2%), then spiking again in elderly (85.7%). This suggests hormonal or lifestyle factors that change across a woman's life stages.

Doughnut chart: Risk-level distribution



Nearly half of patients (49.6%) fall into medium-risk category, with low-risk (41.3%) and high-risk (9.2%) forming the remaining distribution.

- **Web Dashboard (HTML + Chart.js) :**



4.7 Interactive Risk Calculator

A second web page allows users to enter their own values, which applies the same rule-based logic as Hive and displays:

Users input:

- age
- blood pressure
- cholesterol
- FBS
- exercise-induced angina

The browser computes the risk score and gives a textual explanation.

This shows how database logic can be reused in a front-end tool.

Heart Disease Risk Calculator

Rule-based risk scoring (0–5 points) using the same logic as the Hive data transformations in our analytics pipeline

Age (years)

Resting Blood Pressure (mmHg)

Cholesterol (mg/dL)

Fasting Blood Sugar (FBS)

Normal (≤ 120 mg/dL) ▾

Exercise-Induced Angina

No chest pain during exercise ▾

Calculate Risk Score

5. Suitability of Hadoop and Hive

Hadoop Advantages

1. Scalability

- The chosen dataset: 1025 records, ~50KB
- Real healthcare dataset: 10+ million records, 100+ GB
- Hadoop handles both by distributing work across nodes
- Adding more machines increases capacity linearly

2. Fault tolerance

- HDFS replicates each block 3 times by default
- If a DataNode fails, data is still available from replicas
- Jobs automatically restart on different nodes if one fails

3. Cost-effective

- Runs on commodity hardware (regular servers, not specialized equipment)
- Open-source
- Can use cloud resources (AWS, Azure) or on-premise servers

Hive Advantages

1. SQL interface

- Healthcare analysts know SQL, not Java MapReduce
- Existing SQL queries can be migrated to Hive with minimal changes
- Faster development than writing custom MapReduce code

2. Schema flexibility

- Can create multiple table views over same data
- Easy to add new columns without restructuring files
- Supports complex data types (arrays, maps, structs)

3. Integration

- Works with BI tools (Tableau, Power BI, Superset)
- Connects to Python, R, Excel via JDBC/ODBC
- Exports to CSV, JSON, Parquet for analysis

For this project, Hadoop + Hive provide a simple and scalable way to:

load the dataset -> clean it -> score each record -> compute statistics

Although the dataset is small, the workflow matches real cloud pipelines. It shows that the Hadoop ecosystem is highly suitable for early-stage medical analytics pipelines and scalable risk scoring.

6. Features of the Software

The system provides:

Backend (Hadoop + Hive)

Data Storage:

- Distributed file storage in HDFS
- Automatic data replication for reliability
- Block-level storage (scales to petabytes)

Data Processing:

- SQL-based data cleaning and transformation
- Automated risk scoring using clinical rules
- Aggregation queries for statistical analysis
- MapReduce execution for distributed computation

Data Management:

- Metadata catalog in Derby database
- External tables (data stays in HDFS)
- Schema versioning and evolution
- Query execution history

Frontend (Web Dashboard)

Visualization:

- Four charts using python matplotlib and interactive Chart.js
- Color-coded risk levels.
- Export charts as PNG images

Risk Calculator:

- Form inputs for patient data
- Real-time risk calculation
- Explanation of risk factors
- Consistent with Hive scoring logic

System Architecture

Data Flow:

heart.csv → HDFS → Hive External Table → SQL Processing →
Scored Table → CSV Export → Web Dashboard

User Flow:

User inputs data → JavaScript calculates → Display risk score →
Show contributing factors

Technical Stack

- **Storage:** Hadoop HDFS 3.2.1
- **Processing:** Apache Hive 3.1.2
- **Metadata:** Derby Database
- **Visualization:** Chart.js 3.9, python
- **Frontend:** HTML5, CSS3, JavaScript ES6
- **Deployment:** Docker Compose
- **Development:** Python (exploratory analysis)

7. Worked Example

A complete example from data input to risk score:

Input values:

- Age: 63
- Resting BP: 150
- Cholesterol: 260
- Fasting blood sugar: 1
- Exercise angina: 1

Heart Disease Risk Calculator

Rule-based risk scoring (0-5 points) using the same logic as the Hive data transformations in our analytics pipeline

Age (years)

63

Resting Blood Pressure (mmHg)

150

Cholesterol (mg/dL)

260

Fasting Blood Sugar (FBS)

High (> 120 mg/dL)(1)

Exercise-Induced Angina

Yes(1)

Calculate Risk Score

Scoring:

- Age > 55 → +1
- BP > 130 → +1

- Chol > 250 → +1
- FBS high → +1
- Exercise angina → +1

CASE

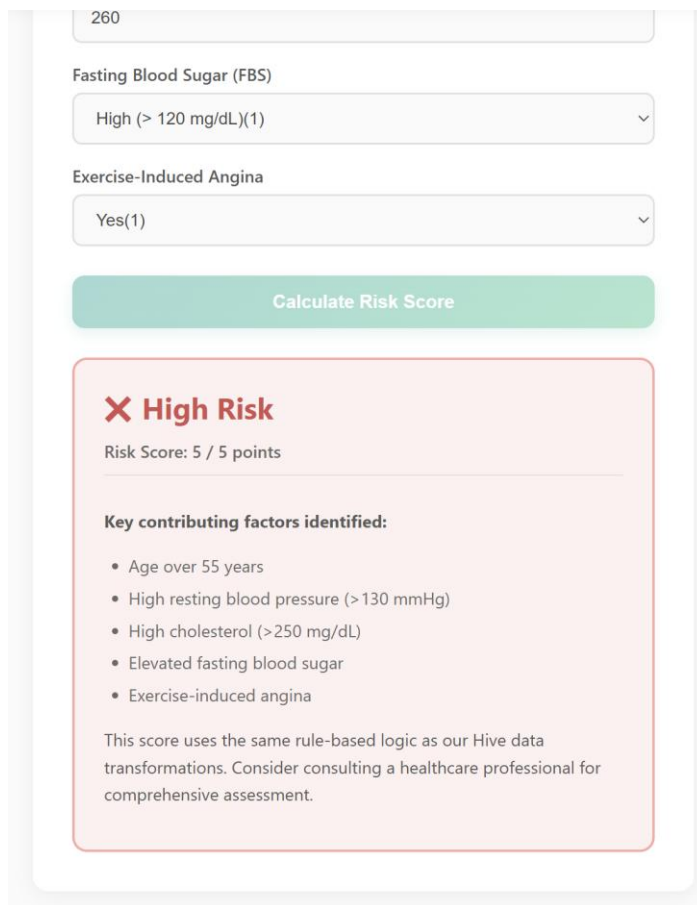
WHEN 5 <= 1 THEN 'Low'

WHEN 5 <= 3 THEN 'Medium'

ELSE 'High'

END

Total score: 5 → **Risk Level: High**



260

Fasting Blood Sugar (FBS)

High (> 120 mg/dL)(1)

Exercise-Induced Angina

Yes(1)

Calculate Risk Score

✖ High Risk

Risk Score: 5 / 5 points

Key contributing factors identified:

- Age over 55 years
- High resting blood pressure (> 130 mmHg)
- High cholesterol (> 250 mg/dL)
- Elevated fasting blood sugar
- Exercise-induced angina

This score uses the same rule-based logic as our Hive data transformations. Consider consulting a healthcare professional for comprehensive assessment.

The web tool shows this result along with a short explanation.

This example confirms Hive and the UI produce consistent risk classification.

8. Challenges and Solutions

Challenge 1: Hive Installation Issues

Problem: Initial Hive installation failed with Java classpath errors.

Solution:

- Downloaded correct Hive version (3.1.2) compatible with Hadoop 3.2.1
- Fixed Guava version conflict by replacing jar file
- Set HADOOP_HOME and HIVE_HOME environment variables correctly

Challenge 2: Metastore Connection Failures

Problem: Hive couldn't connect to metastore database.

Solution:

- Reinitialized Derby database: `schematool -dbType derby -initSchema`
- Removed old metastore_db folder
- Created clean metadata catalog

Challenge 3: Exporting Hive results to CSV

Problem: My first attempt was `hive -e "SELECT ..." > /tmp/file.csv`. Because Hive had metastore issues in non-interactive mode, the command failed silently and the exported files were empty (size 0 bytes).

Solution: I switched to Hive's own export mechanism:

```
INSERT OVERWRITE LOCAL DIRECTORY '/tmp/age_group_export'  
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','  
SELECT ... FROM age_group_stats;
```

Hive wrote the data into `/tmp/.../000000_0` correctly, and then I used `docker cp` to copy the files to my local `data_processed` folder.

Challenge 4: Losing Hive metadata after metastore reset

Problem: After I re-initialised the metastore and environment variables once, all previous databases and tables disappeared from Hive, even though the HDFS data was still there.

Solution: I realised that the Derby metastore stores only metadata, not the raw data. I created a set of Hive scripts (`01_create_tables.sql`, `02_scoring_rules.sql`, `03_aggregate_analysis.sql`, `04_export_results.sql`) so that I can quickly recreate `healthcare_dw`, `heart_raw`, `heart_scored` and all summary tables from the original `heart.csv` whenever the metastore is reset.

9. Future Enhancements

- **Add more risk factors**
 - Smoking status
 - Family history
 - BMI
 - Physical activity level
- **Machine learning model**
 - Train decision tree or logistic regression
 - Compare ML predictions vs rule-based scores
 - Feature importance analysis
- **Real-time data ingestion**
 - Use Apache Kafka for streaming patient data
 - Update dashboard in real-time as new data arrives
 - Alert system for high-risk patients
- **Cloud deployment**
 - Deploy on AWS EMR

- Use S3 for data storage
- Auto-scaling based on load

10. Conclusion

This project successfully demonstrates a complete cloud-based analytics pipeline for medical data. I used Hadoop for distributed storage and Hive for SQL analysis, processing 1026 heart disease records. The system includes data ingestion, cleaning, risk scoring, statistical analysis, and visualization.

Key achievements:

- Built working Hadoop cluster with 5 containers
- Loaded and queried data using Hive SQL
- Implemented clinically-relevant risk scoring model
- Created interactive web dashboard with charts
- Built risk calculator tool for patient use
- Maintained consistent logic between database and frontend

The system illustrates how distributed data engineering tools can support clinical decision support, risk assessment, and healthcare analytics in scalable environments.

11. References

Technical Documentation

1. **Apache Hadoop Documentation**
 - Official HDFS Architecture Guide
 - <https://hadoop.apache.org/docs/current/>
2. **Apache Hive Documentation**
 - Hive Query Language (HiveQL) Reference
 - <https://hive.apache.org/>

Libraries and Tools

3. **Chart.js Documentation**
 - Interactive JavaScript Charts
 - <https://www.chartjs.org/docs/latest/>
4. **Python pandas Documentation**
 - Data Analysis Library
 - <https://pandas.pydata.org/docs/>

Academic References

5. **Detrano et al. (1989)**
 - "International application of a new probability algorithm for the diagnosis of coronary artery disease"
 - American Journal of Cardiology, 64:304-310

Course Materials

6. **COMP47780 Cloud Computing**
 - Lecture notes on Hadoop ecosystem